5/9 Gestión de Productos de Software (debrief)

Resumen Ejecutivo

Propósito y enfoque. Un producto de software es “conocimiento empaquetado” (código, configuraciones, documentación y experiencia de usuario) creado para resolver necesidades reales. Para gestionarlo efectivamente, conviene adoptar **ciclos iterativo-incrementales** con base **empírica** (inspección y adaptación continua), evitando planes rígidos que cristalizan supuestos.

1) Fundamentos y calidad “viable”

- **Proceso → Proyecto → Producto**: tres niveles que conviene alinear con un **ciclo de vida** adecuado (iterativo, evolutivo o en cascada).
- **Piramidalidad de calidad** para validar hipótesis sin sesgos:

  Funcionalidad → Fiabilidad → Usabilidad → Diseño.

  Si falta fiabilidad o usabilidad, el rechazo del usuario puede deberse a la mala experiencia y no a la propuesta de valor.
- **Valor vs. desperdicio (Lean)**: la mayoría del uso real se concentra en pocas capacidades; desarrollar “extra” sin evidencia es muda. Priorizar lo **que el usuario hará** (comportamiento), no lo que “podría querer” (opiniones).

2) Modelo de “mínimos” para aprender y crecer

- **MVP (Minimum Viable Product)**: versión mínima para **aprender rápido** sobre problema-cliente-solución con **usuarios reales** (early adopters). No es “producto final” ni “mínimo de calidad”: es lo mínimo **viable para producir aprendizaje válido**.

  Señales de éxito: uso/pago inicial, retención incipiente, feedback convergente que indica “qué construir después”.
- **MVF (Minimum Viable Feature)**: **rebanada** funcional acotada para validar **una hipótesis puntual** (utilidad/adopción) con muy baja fricción y telemetría explícita.
- **MMF (Minimal Marketable Feature)**: paquete liberable con **valor de negocio** cuando la zona de oportunidad ya está validada.
- **MMP / MMR**: conjunto mínimo listo para **mercado/producción**; el **MMR** es el release mínimo que entrega valor nuevo, y el **MMP** suele ser su primera iteración pública.

> Errores comunes: confundir MVP (aprender) con MMF/MMP (ganar/monetizar); sobre-enfatizar “mínimo” y olvidar “viable”; no iterar tras el primer MVP.

3) Bucle Build–Experiment–Learn (BEL)

1. **Idea → Hipótesis** (de valor o crecimiento).
2. **Construir** el **mínimo** (MVP/MVF) que cubra un flujo **end-to-end**.
3. **Experimentar** con usuarios objetivo y **métricas accionables** (no de vanidad).

4. **Aprender**: **perseverar / ajustar / pivotear** y realimentar el backlog.
5. **Evolucionar**: escalar con **MMF**, consolidar con **MMR/MMP**.

4) Priorización y roadmap

- Usar la **matriz Importante × Urgente** para el contenido del MVP/MVF:

  Hacer (hipótesis críticas) · **Planear** (relevantes, no urgentes) · **Delegar** (accesorios) · **Eliminar** (ruido).
- **Roadmap adaptativo**: 30-60-90 días con hitos de aprendizaje (no solo entregables). El criterio de avance es **decisiones tomadas con evidencia**, no la cantidad de funcionalidades.

5) Estimaciones, métricas y gobernanza ligera

- **Estimaciones ágiles**: dimensionar en **tamaño relativo** (Story Points), no en horas; medir **Velocity** solo con trabajo "Done". El objetivo es **prever y conversar**, no "prometer".
- **Métricas mínimas**: adopción (activación/retención), tiempo a primer valor, tasa de conversión por flujo, defectos bloqueantes, % dentro de SLA.
- **Políticas y riesgos**: definir criterios de privacidad/seguridad desde el MVP; mitigar riesgos de adopción con pilotos y UX simple; evitar "sobrediseño" prematuro (YAGNI).

Idea central

La **gestión de productos** es un **sistema de aprendizaje guiado por datos**: empezar pequeño (MVP/MVF), medir comportamiento real, decidir con evidencia y escalar con MMF/MMR/MMP. El éxito no es "entregar software", sino **lograr que los usuarios cambien su comportamiento porque el producto les resuelve un problema mejor que sus alternativas**.

---

1. Fundamentos de la Creación de Productos de Software.

🔥 PROCESO ⟶ PROYECTO ⟶ PRODUCTO DE SW tiene un =

= Ciclo de vida - Iterativo Incremental / en cascada / evolutivo

¿Por qué creamos productos de software?

La creación de productos de software responde a diversas motivaciones clave, tanto para las organizaciones como para los usuarios finales. En esencia, **un producto de software es un artefacto construido para satisfacer una necesidad específica basada en ciertos requisitos.**

[Síntesis] Un producto de software va más allá del simple código; es el "conocimiento empaquetado" que incluye programas, archivos de configuración, documentación del sistema y del usuario, y otros elementos necesarios para su correcto funcionamiento. Comprender su naturaleza y sus atributos de calidad es fundamental para su gestión.

Las razones principales incluyen:

- **Satisfacer a los clientes:** Esta es una motivación fundamental. El objetivo primordial de la ingeniería de software es entregar un producto de calidad que satisfaga las expectativas del cliente. En los enfoques ágiles, la máxima prioridad es la satisfacción del cliente a través de la entrega temprana y continua de software valioso.
- **Tener muchos usuarios logueados:** Esto se relaciona con el éxito de adopción y uso del producto, lo cual a menudo es un indicador de que el producto está brindando valor.

- **Obtener mucho dinero:** La rentabilidad es un objetivo empresarial claro para la creación de productos. Un producto exitoso no solo genera valor para el cliente, sino también ganancias para la organización desarrolladora.
- **Realizar una gran visión, cambiar el mundo:** Más allá de los beneficios económicos, muchos productos aspiran a tener un impacto significativo, transformar industrias o resolver problemas globales.

Es importante destacar que el valor real de un producto de software a menudo reside en una pequeña fracción de sus características. Aproximadamente solo el *7% de las características de un software son usadas consistentemente*, concentrando la mayor parte del valor agregado. Crear funcionalidades en exceso que no aporten valor se considera un **desperdicio** desde la filosofía Lean. Por lo tanto, el enfoque debe estar en construir el **producto correcto** que agregue valor al negocio y facilite la vida de los usuarios.

Características clave de un producto de software y su evolución

La evolución de un producto de software puede entenderse como una jerarquía de características, desde las más básicas y fundamentales hasta aquellas que ofrecen una experiencia superior al usuario. Esta progresión refleja los **factores de calidad** que un software debe poseer.

La pirámide de evolución de productos de software (de abajo hacia arriba) incluye:

- **Funcional (Útil):** Es la base. El software debe cumplir con su especificación y los objetivos propuestos por el cliente. Sin esta cualidad, el software no cumple su propósito principal.
- **Confiable:** El grado en que se espera que el programa realice su función con la precisión requerida. Un software debe ser estable y consistente en su operación.
- **Usable (Facilidad de uso):** El esfuerzo necesario para aprender, operar y preparar datos de entrada, e interpretar las salidas de un programa. Un diseño intuitivo y una buena experiencia de usuario son clave aquí.
- **Conveniente:** El software no solo funciona bien, sino que también es fácil de acceder y usar en el contexto del usuario.
- **Placentero:** Va más allá de la mera utilidad, ofreciendo una experiencia agradable o satisfactoria al usuario. Esto implica una atención al diseño estético y a la fluidez de la interacción.

- **Significativo:** En el nivel más alto, el software aporta un valor profundo o un impacto transformador al usuario o a la organización. Esto se alinea con la visión de negocio y el propósito fundamental del producto. Suelen ser impactos innovadores en la sociedad, vienen y cambian rotundamente el comportamiento de un individuo ante una tarea comun, como la comunicacion a distancia(whatsapp) la ubicacione geografica por mapas (google maps)

Esta jerarquía resuena con los **Factores de Calidad de McCall**, que clasifican los atributos del software en torno a la operación, revisión y transición del producto. Las características básicas (Funcional, Confiable) se relacionan con la operación, mientras que las características superiores (Usable, Conveniente, Placentero, Significativo) abordan la experiencia y el valor estratégico.

Para las organizaciones, cruzar la "grieta" entre un software meramente funcional y uno que sea realmente **significativo** y **focalizado en experiencias (gente, actividades, contexto)** es un desafío considerable. Implica ir más allá de "que solo funcione" para aportar un valor agregado real al negocio del usuario.

2. Herramientas de Gestión de Productos Mínimos en Contextos Lean/Agile

En la gestión de productos, especialmente en enfoques Lean y Ágil, se utilizan diferentes "productos mínimos" para validar ideas, gestionar el crecimiento y satisfacer necesidades específicas. Estos conceptos son fundamentales para el desarrollo iterativo e incremental del software.

Hipótesis de Valor Único y el Producto Mínimo Viable (MVP)

Todo producto nuevo se inicia con una **hipótesis de valor único**; la idea de que el producto o servicio será distintivo en el mercado. Para validar esta hipótesis de la manera más eficiente, se crea un **Producto Mínimo Viable (MVP)**.

- **Definición de MVP:** Es la versión de un producto nuevo creada con el **menor esfuerzo posible**, dirigida a un subconjunto de clientes potenciales y utilizada para obtener **aprendizaje validado**. Su enfoque principal es probar la propuesta de valor única, pero también incluye algunas funciones de "Table stakes" (básicas y esperadas) para asegurar que sea "viable" como producto.
- **El MVP como hipótesis:** Un MVP es, en sí mismo, una hipótesis. Puede ser lo suficientemente bueno para encontrar un mercado o puede que no. Su objetivo es **probar hipótesis comerciales fundamentales**.
- **Errores comunes al implementar un MVP:**
  - **Confundir MVP con MMF o MMP:** Un MVP se enfoca en el **aprendizaje**, mientras que una Característica Mínima Comercializable (MMF) o un Producto Mínimo Comercializable (MMP) se enfocan en "ganar" o generar valor de negocio directamente. Mezclarlos puede llevar a entregar algo que no satisface las necesidades del cliente.
  - **Ignorar la parte "viable":** Enfatizar solo lo "mínimo" sin asegurar que el producto tenga la calidad suficiente para una evaluación precisa por parte de los clientes. Un MVP de baja calidad no permitirá obtener retroalimentación útil.
  - **Tratar el MVP como producto final:** Entregar el MVP y luego no realizar más cambios, ignorando el feedback recibido, es un error.
- **Principios clave para el MVP:**
  - **Piensa en grande, pero comienza en pequeño:** Define una visión ambiciosa, pero ejecuta un primer paso pequeño y enfocado.
  - **Requiere juicio y simplificación:** Decidir qué tan complejo debe ser un MVP no es una fórmula, sino que requiere juicio. Ante la duda, simplificar es la mejor opción. Evitar la construcción

excesiva y las promesas exageradas es crucial.

  - **Evitar el desperdicio:** Cualquier trabajo adicional que vaya más allá de lo necesario para iniciar el ciclo de aprendizaje podría ser un desperdicio. Esto se alinea con la filosofía Lean de maximizar el valor con el mínimo consumo de recursos.
  - **Enfoque de "solo lo suficiente":** En el desarrollo ágil de requerimientos, se busca iniciar con lo mínimo necesario para generar retroalimentación y avanzar continuamente.

- **Feedback del MVP:**
  - **Feedback divergente:** Si, después de interactuar con clientes potenciales, cada uno te pide una característica "X" diferente ("Esto es genial, pero para usarlo necesito X"), indica que aún no se ha encontrado un mercado claro para el producto.
  - **Feedback convergente:** Si, por el contrario, las respuestas apuntan al **MISMO "X"**, tiene sentido revisar la hipótesis de Cliente/Problema/Solución, lo que sugiere un camino claro para el desarrollo futuro. Este feedback es vital para la adaptación del proceso.

Gestión de Productos y Crecimiento Post-MVP

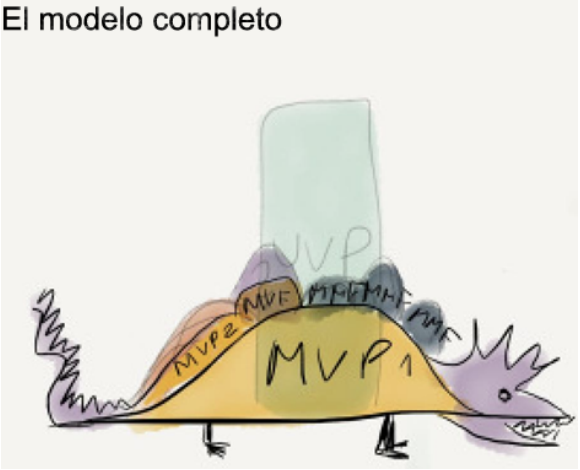
Una vez que se ha recopilado feedback del MVP y se ha realizado investigación de gestión de productos, se identifican áreas con potencial de crecimiento. Estas áreas pueden tanto **ampliar la propuesta de valor única** como hacer el **producto actual más robusto**.

Aquí es donde entran otros conceptos de productos mínimos:

- **Característica Mínima Comercializable (MMF - Minimal Marketable Feature):**
  - **Definición:** En áreas con una fuerte indicación de valor, se puede definir directamente un MMF. Su objetivo es encontrar la pieza mínima que pueda empezar a **traer crecimiento**.
  - **Enfoque:** A diferencia del MVP, el MMF se enfoca en "ganar" y aportar valor, suponiendo una alta certeza de la existencia de dicho valor y de cómo entregarlo. Es posible dividir una característica grande en MMFs más pequeños.
- **Característica Mínima Viable (MVF - Minimal Viable Feature):**
  - **Definición:** Un MVF debe proporcionar un valor claro a los usuarios y ser fácil de usar. Requiere recursos mínimos y su diseño y confiabilidad deben guiarse por los estándares de calidad de la industria y la producción.
  - **Usuarios objetivo:** Se dirige a un grupo de usuarios flexibles y tolerantes, como los primeros adoptantes, clientes leales o miembros de una junta asesora de clientes.
  - **Uso estratégico:** Los resultados de un MVF ayudan a tomar decisiones estratégicas sobre el producto.

El Modelo Completo: Relación entre MVP, MMF, MMP, MMR

!!IMPORTANTE!!



El modelo de productos mínimos para la gestión de productos es un **enfoque iterativo para construir y evolucionar software**. Nos basamos en la **generación de valor como fundamento BASE para hacer el MVP** y todo lo que le sigue.

MVP (Producto Mínimo Viable): Es la **primera versión mínima** de un producto, creada con el **menor esfuerzo** para **aprender** rápido y **validar la hipótesis de valor** (Lean Startup).

Para quién: un **subconjunto acotado de clientes** (early adopters) que nos ayude a probar supuestos clave.

Para qué: obtener **aprendizaje validado** observando **comportamiento real** (uso/compra), no intenciones.

Qué NO es: no es "lo mínimo de calidad"; es lo **mínimo viable para aprender**. Si no permite medir aprendizaje real, **no es** un MVP.

Señales de éxito:

- **Valor suficiente** como para que la gente lo **use o pague** al inicio.
- **Beneficio percibido** que **retenga** a los primeros usuarios.
- **Ciclo de feedback** claro que guíe qué construir después.

En resumen:

- Versión de un nuevo producto creado con el menor esfuerzo posible
- Dirigido a un subconjunto de clientes potenciales
- Utilizado para obtener aprendizaje validado.
- Más cercano a los prototipos que a una version real funcionando de un producto

---

MVF (Característica Mínima Viable): Es una **característica mínima viable** (una *feature* pequeña) que se construye con el **menor esfuerzo** posible para **validar** una hipótesis concreta con **usuarios reales**. Es como un **MVP en miniatura** pero focalizado en **una sola capacidad**.

Para quién: un **grupo acotado y tolerante** (early adopters, clientela leal, panel asesor) dispuesto a probar y dar feedback rápido.

Para qué: medir **utilidad y adopción** de esa característica específica y decidir:

- **Hit gold** → **expandir** esa zona del producto con **MMF** (más alcance "marketable").
- **No despega** → **ajustar/pivotar** la idea o **abandonar** y probar otra vía de crecimiento.

Cómo luce:

- Alcance **muy limitado** (una tarea bien definida de principio a fin).
- **Valor claro** para el usuario y **uso simple** (fricción mínima).

- **Instrumentado** para observar comportamiento (eventos, funnels, tasa de uso/retención).

Qué NO es:

- No es una *prueba técnica aislada* sin valor para el usuario.
- No es “lo mínimo de calidad”; es lo **mínimo que permite aprender** sin sesgos.

Relación con MVP/MMF:

- **MVF** = “**rebanada finita**” para validar una hipótesis puntual.
- **MMF** = **paquete marketable** (listo para mercado) que puede **nacer** de un MVF exitoso.
- **MVP** = validación a **escala de producto**; el MVF es su **átomo funcional**.

En Resumen:

- MVF es una versión mini del MVP.
- Característica a pequeña escala que se puede construir e implementar rápidamente, utilizando recursos mínimos, para una población objetivo para probar la utilidad y adopción de la característica.
- Un MVF debe proporcionar un valor claro a los usuarios y ser fácil de usar.
  diseño y la confiabilidad.
- MVF requiere recursos mínimos, los estándares de calidad de la industria y la producción deben guiar el
- El grupo de usuarios para un MVF, son los primeros en adoptar, los clientes leales que han compartido conocimientos anteriormente o los miembros de una junta asesora de clientes.. Usuarios flexibles y tolerantes.
- Los resultados le ayudarán a tomar decisiones estratégicas sobre productos

---

MMF (Característica Mínima Comercializable): Una característica diseñada para generar crecimiento, con un enfoque en el "ganar" una vez que el valor es más claro.
El objetivo del MMF es aportar valor. Supone una alta certeza de que existe valor en esta área y que sabemos cuál debe ser el producto para proporcionar este valor. La **hipótesis se centra en una característica en lugar del producto.**

- es la pieza más pequeña de funcionalidad que puede ser
  liberada
- tiene valor tanto para la organización como para los usuarios.
- Es parte de un MMR or MMP

MMP (Producto Mínimo Comercializable): Un producto con el conjunto mínimo de características que lo hacen listo para el mercado y orientado a la obtención de valor comercial. Version de producto listo para PRODUCCION. Todo el mundo habla del MVP cuando en realidad se refiere al MMP, y si el MMP sale a produccion con una caracteristica se lo denota MMF no MMP.

- Primer release de un MMR dirigido a primeros usuarios (early
  adopters)
- Focalizado en características clave que satisfarán a este grupo
  clave.

MMR (Características Mínimas del Release): Se refiere a las características mínimas necesarias para una entrega o lanzamiento específico del producto.

- Release de un producto que tiene el conjunto de características más pequeño posible.

- El incremento más pequeño que ofrece un valor nuevo a los usuarios y satisfice sus necesidades actuales.

> 🚨 MMP = MMR1

- **UVP (Producto Unico Viable). Es la motivación original, ideal y utópico, ya que hoy en dia no hay sw que no evolucione con el tiempo.**

Esta relación subraya una progresión lógica en el desarrollo de productos: se empieza por aprender y validar con el menor riesgo (MVP, MVF), y a medida que se obtiene conocimiento y se reduce la incertidumbre, se avanza hacia la entrega de valor comercial y el crecimiento (MMF, MMP, MMR).

> 💡 En mercados inciertos, el producto se "cultiva" probando varios **MVP**. Cuando aparece el ajuste producto–mercado, se combinan **MMF** y **MVF** según la incertidumbre del negocio/requisitos en las áreas foco. Aunque **MVP/MMF/MVF** son atómicos para el negocio, pueden resultar grandes de implementar; por eso se aplica el **"dinosaurio carpaccio"**: cortar cada pieza en porciones pequeñas (**User Stories**) para reducir riesgo de ejecución/tecnología. Estas porciones pueden o no tener valor comercial directo, pero permiten obtener **retroalimentación temprana** sobre decisiones a lo largo del camino.

Errores comunes (MVP vs. MMF/MMP)

- **Confundir** MVP (aprendizaje) con MMF/MMP (enfoque en **"ganar/monetizar"**).
- **Sobre-enfatizar lo "mínimo"** y olvidar lo **"viable"** (calidad insuficiente para evaluar adopción real).
- **No iterar** tras el primer MVP, ignorando comentarios de usuarios.

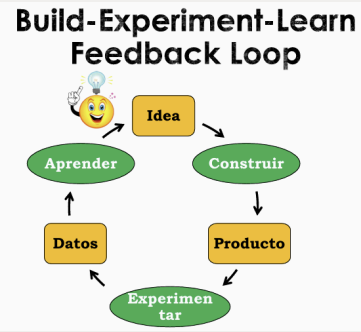
> ♻️ **Valor vs. Desperdicio.**
>
> *Lean Thinking define la creación de valor como proveer beneficios a los clientes, cualquier otra cosa es desperdicio!*
>
> La productividad de un Startup **no puede medirse en términos de cuánto se construye cada día, por el contrario se debe medir en términos de averiguar la cosa correcta a construir cada día**

Hacia la creación de valor

¿Qué es el Build–Experiment–Learn?

Es un **ciclo corto** para validar decisiones de producto con usuarios reales:

1. **Idea** → formular una **hipótesis** (de valor o de crecimiento).
2. **Construir** → crear el **mínimo** (MVP/MVF) que permita probarla.
3. **Experimentar** → exponer a usuarios objetivo con un **diseño de experimento** claro.
4. **Datos** → recolectar **métricas accionables** (no vanidad).
5. **Aprender** → decidir: **perseverar, ajustar o pivotear**… y alimentar la **siguiente idea**.



El éxito no **es entregar un producto**, el éxito se trata de entregar un producto (o característica de producto) **que el cliente usará.**
La forma de hacerlo es **alinear los esfuerzos continuamente hacia las necesidades** reales de los clientes.
The Build-Experiment-Learn feedback loop permite descubrir las necesidades del cliente y alinearlas metodológicamente.

Vínculos con otras unidades: Ingeniería de Software, Calidad y Procesos

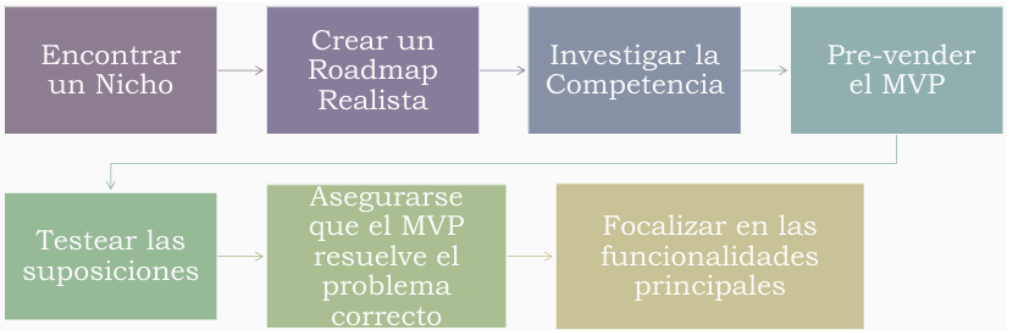
Estos conceptos de gestión de productos mínimos están intrínsecamente ligados a varias disciplinas de la ingeniería de software y la gestión de proyectos:

- **Ingeniería de Software:** Como disciplina, se ocupa de todos los aspectos de la producción de software, desde la especificación hasta el mantenimiento. Los productos mínimos son herramientas para gestionar el ciclo de vida del software de manera eficiente y centrada en el valor.
  - El software se define como un conjunto de programas y la documentación que los acompaña, incluyendo archivos de configuración, documentación de estructura y manuales de usuario. Puede ser de tipo genérico (para el mercado abierto) o personalizado (a medida para un cliente).
- **Filosofía Lean y Agile:** Estos enfoques son la base para el uso de MVPs y MMFs.
  - **Lean:** Busca maximizar el valor para el cliente con el mínimo consumo de recursos, identificando y eliminando desperdicios. La creación de características "extra" o la especificación anticipada de requisitos obsoletos son ejemplos de desperdicio que los productos mínimos ayudan a evitar.
  - **Agile:** Es una ideología con principios que guían el desarrollo de productos. Prioriza individuos e interacciones, software funcionando sobre documentación extensa, colaboración con el cliente y respuesta al cambio. La entrega temprana y continua de software valioso es la máxima prioridad. Los MVPs y MMFs son manifestaciones de estos principios, permitiendo la adaptación a cambios y la entrega incremental de valor.

    - **Empirismo:** Tanto Scrum como Kanban se basan en el empirismo, utilizando ciclos cortos de inspección y adaptación para el aprendizaje y la mejora continua. Los MVPs son fundamentales para este ciclo de retroalimentación.
- **Gestión de Proyectos:** Los productos mínimos influyen en la definición del **alcance del producto** (todas las características) y el **alcance del proyecto** (el trabajo necesario para entregar el producto).
    - **Estimaciones:** Las estimaciones en entornos ágiles se centran en el tamaño de las historias de usuario (Story Points), el esfuerzo y la incertidumbre, siendo realizadas por el propio equipo. Los productos mínimos ayudan a refinar estas estimaciones con el tiempo al tener un enfoque incremental.
    - **Riesgos:** La creación de MVPs ayuda a mitigar riesgos al validar hipótesis con el menor esfuerzo posible.
- **Calidad de Software y Aseguramiento de Calidad (QA):**
    - Aunque el MVP busca ser mínimo, la parte "viable" implica una **calidad suficiente** para que el cliente pueda evaluarlo correctamente.
    - **QA como medida preventiva:** El aseguramiento de calidad define procesos y estándares que conducen a productos de alta calidad y actúa como medida preventiva. Para las MVF, se deben seguir los estándares de calidad de la industria.
    - **Métricas de Producto:** Las métricas enfocadas en el producto (ej. defectos por severidad, cantidad de líneas de código) son responsabilidad del equipo de desarrollo y Testing, y buscan generar indicadores en tiempo real de la eficacia del análisis, diseño, código y calidad general del software.
- **Requerimientos Ágiles y User Stories:** Los MVPs y MMFs se construyen a partir de requerimientos ágiles, que se expresan a menudo como User Stories.
    - Las **User Stories** son descripciones cortas de una necesidad del usuario, escritas en un formato simple (Como `<rol>` yo puedo `<actividad>` de forma tal que `<valor de negocio>`).
    - Los **Criterios de Aceptación** son parte esencial de las User Stories, definiendo los límites de la funcionalidad y facilitando las pruebas. Se escriben a nivel de usuario y son objetivos y verificables.
    - Las **Spikes** son un tipo especial de User Stories utilizadas para eliminar el riesgo e incertidumbre, permitiendo familiarizarse con nuevas tecnologías o analizar comportamientos complejos antes de dividir una historia en piezas manejables.
    - La comunicación cara a cara y la colaboración entre el Product Owner y el equipo son pilares fundamentales para la definición y el refinamiento continuo de los requerimientos en entornos ágiles.

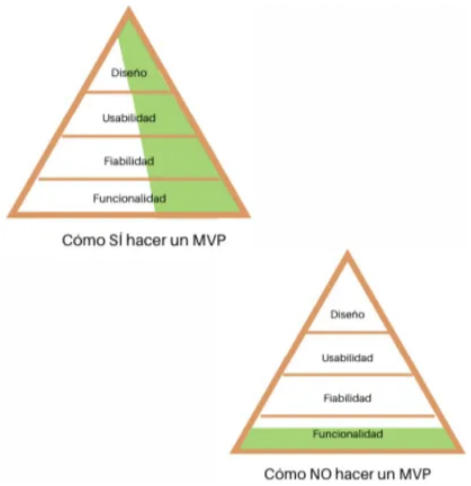
Preparar un MVP.

Un MVP surge de un **encuadre estratégico** previo: (a) **nicho** claramente delimitado (quién y para qué), (b) **roadmap realista** que acota el alcance al aprendizaje esencial, (c) **inteligencia competitiva** para ubicar la propuesta en el mapa de alternativas, y (d) **pre-venta** como señal temprana de intención. Con ese encuadre, la teoría Lean exige **testear suposiciones** (hipótesis de valor y de crecimiento), **verificar que el problema sea el correcto** (evitar soluciones en busca de problema) y **focalizar en funcionalidades principales**: lo mínimo que permita observar comportamiento real y extraer evidencia.



Qué hace “viable” a un MVP.

La viabilidad no se reduce a “tener funciones”.
La pirámide correcta es **Funcionalidad → Fiabilidad → Usabilidad → Diseño**.



- **Funcionalidad**: cubre de punta a punta un uso significativo.
- **Fiabilidad**: estabilidad suficiente para que la evidencia no se vea sesgada por fallas.
- **Usabilidad**: fricción baja; permite llegar al “primer valor” sin esfuerzo.
- **Diseño**: refuerza comprensión y deseo de uso. Invertir la pirámide (solo “funcionalidades” sin soporte de calidad) **invalida el aprendizaje**: no sabés si el rechazo proviene de la propuesta o de la mala experiencia.

Priorizar teoría del MVP.

La **matriz Importante×Urgente** ordena el contenido del MVP:

1. **Hacer** (importante+urgente): hipótesis críticas que definen la validez del modelo.

2. **Planear** (importante+no urgente): elementos relevantes pero postergables a un siguiente release.
3. **Delegar** (no importante+urgente): accesorios o integraciones que no cambian el aprendizaje central.
4. **Eliminar** (no importante+no urgente): ruido que diluye señal y complica la interpretación de resultados.

Idea central

Un **MVP** es un **experimento de mercado con calidad suficiente** para producir **aprendizaje válido**. Se prepara desde el posicionamiento (nicho/competencia), se **limita** a lo esencial (piramidalidad de calidad) y se **prioriza** por valor informativo, no por cantidad de funciones.